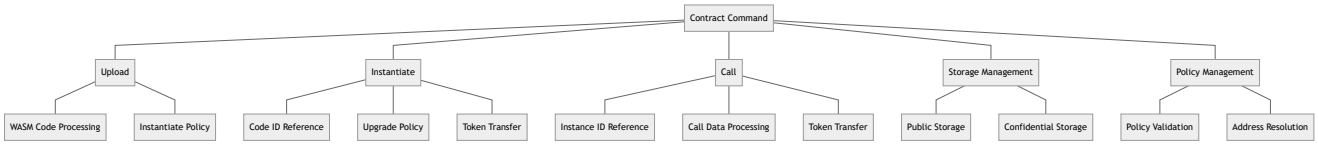
스마트 컨트랙트 기능 분석 문서

개요

Oasis CLI의 스마트 컨트랙트 기능은 WebAssembly(WASM) 기반 스마트 컨트랙트의 전체 생명주기를 관리합니다. 이 문서는 컨트랙트 업로드, 인스턴스화, 호출, 스토리지 관리, 정책 관리 및 업그레이드 과정의 구현을 상세히 분석합니다.

아키텍처 개요

컨트랙트 시스템 구조



핵심 구성 요소

1. **Contract Command Handler** (`cmd/contract.go`)

- 모든 컨트랙트 관련 명령어의 진입점
- 하위 명령어들의 조정 및 관리

2. **Transaction Processing**

- 컨트랙트 트랜잭션 생성 및 서명
- 가스 추정 및 수수료 계산

3. **Storage Interface**

- 컨트랙트 스토리지 조회 및 덤프
- 공개/기밀 스토리지 구분 처리

4. **Policy Management**

- 인스턴스화 정책 관리
- 업그레이드 정책 관리

컨트랙트 업로드 과정

1. WASM 코드 처리

```
// WASM 파일 읽기
wasmData, err := os.ReadFile(wasmFilename)
cobra.CheckErr(err)


// 코드 압축 처리
Code: contracts.CompressCode(wasmData),
```

주요 특징:

- WASM 바이너리 파일을 직접 읽어서 처리
- 자동 압축을 통한 스토리지 최적화
- ABI 버전 지정 (현재 ABIOasisV1 사용)

2. 인스턴스화 정책 설정

```
func parsePolicy(net *config.Network, wallet *cliConfig.Account, policy string
switch {
case policy == "nobody":
return &contracts.Policy{Nobody: &struct{}{}}
case policy == "everyone":
return &contracts.Policy{Everyone: &struct{}{}}
case policy == "owner":
address := wallet.GetAddress()
return &contracts.Policy{Address: &address}
case strings.HasPrefix(policy, "address:"):
// 특정 주소로 제한
policy = strings.TrimPrefix(policy, "address:")
address, _, err := common.ResolveLocalAccountOrAddress(net, policy)
if err != nil {
cobra.CheckErr(fmt.Errorf("malformed address in policy: %w", err))
}
return &contracts.Policy{Address: address}
}
}
```

정책 유형:

- `nobody` : 아무도 인스턴스화할 수 없음
- `everyone` : 모든 사용자가 인스턴스화 가능
- `owner` : 업로드한 사용자만 인스턴스화 가능
- `address:주소` : 특정 주소만 인스턴스화 가능

3. 업로드 트랜잭션 생성

```
tx := contracts.NewUploadTx(nil, &contracts.Upload{
ABI: contracts.ABIOasisV1,
InstantiatePolicy: *instantiatePolicy,
Code: contracts.CompressCode(wasmData),
})
```

트랜잭션 구조:

- ABI 버전 명시
- 인스턴스화 정책 포함
- 압축된 WASM 코드
- 가스 추정 및 수수료 계산

컨트랙트 인스턴스화 과정

1. 인스턴스화 데이터 처리

```
func parseData(data string) interface{} {
var result interface{}
```

```
if len(data) > 0 {
err := yaml.Unmarshal([]byte(data), &result)
cobra.CheckErr(err)
}
return result
}
```

데이터 처리 특징:

- YAML 형식의 초기화 데이터 지원
- 빈 데이터도 허용
- 자동 타입 변환

2. 토큰 전송 처리

```
func parseTokens(pt *config.ParaTime, tokens []string) []types.BaseUnits {
result := []types.BaseUnits{}
for _, raw := range tokens {
amount, err := helpers.ParseParaTimeDenomination(pt, raw, types.NativeDenominati
if err != nil {
cobra.CheckErr(fmt.Errorf("malformed token amount: %w", err))
}
result = append(result, *amount)
}
return result
```

```
}
```

토큰 전송 기능:

- 인스턴스화 시 토큰 전송 지원
- ParaTime별 denomination 처리
- 다중 토큰 타입 지원

3. 업그레이드 정책 설정

```
upgradesPolicy := parsePolicy(npa.Network, npa.Account, contractUpgradesPolicy)
```

```
tx := contracts.NewInstantiateTx(nil, &contracts.Instantiate{
CodeID: contracts.CodeID(codeID),
UpgradesPolicy: *upgradesPolicy,
Data: cbor.Marshal(data),
Tokens: tokens,
})
```

업그레이드 정책:

- 컨트랙트 업그레이드 권한 제어
- 인스턴스화 정책과 동일한 형태
- 기본값은 “owner” (인스턴스 생성자)

컨트랙트 호출 과정

1. 호출 데이터 준비

```
// 호출 인수 파싱
data := parseData(strData)

// 토큰 전송 파싱
tokens := parseTokens(npa.ParaTime, contractTokens)

// 트랜잭션 생성
tx := contracts.NewCallTx(nil, &contracts.Call{
ID: contracts.InstanceID(instanceID),
Data: cbor.Marshal(data),
Tokens: tokens,
})
```

호출 특징:

- YAML 형식의 호출 데이터
- CBOR 직렬화를 통한 데이터 전송
- 호출 시 토큰 전송 지원

2. 결과 처리

```
var  result contracts.CallResult
if !common.BroadcastOrExportTransaction(ctx, npa, conn, sigTx, meta, &result) {
```

```
return
}

fmt.Printf("Call result:\n")

var decResult interface{}
err = cbor.Unmarshal(result, &decResult)
if err != nil {
cobra.CheckErr(fmt.Errorf("failed to unmarshal call result: %w", err))
}

formatted, err := yaml.Marshal(decResult)
cobra.CheckErr(err)
fmt.Println(string(formatted))
```

결과 처리 과정:

- CBOR 역직렬화
- YAML 형식으로 출력
- 에러 처리 및 사용자 친화적 표시

스토리지 관리 시스템

1. 스토리지 종류

```
var  storeKind contracts.StoreKind

cobra.CheckErr(storeKind.UnmarshalText([]byte(contractStorageDumpKind)))


// 지원되는 스토리지 종류

contracts.StoreKindPublicName  // "public"

contracts.StoreKindConfidentialName  // "confidential"
```

스토리지 유형:

- **Public Storage**: 공개적으로 접근 가능한 스토리지
- **Confidential Storage**: 기밀 스토리지 (접근 제한)

2. 스토리지 덤프 기능

```
res, err := conn.Runtime(npa.ParaTime).Contracts.InstanceRawStorage(

ctx,

client.RoundLatest,

contracts.InstanceID(instanceID),

storeKind,

contractStorageDumpLimit,

contractStorageDumpOffset,

)
```

덤프 기능:

- 페이지네이션 지원 (limit, offset)

- 키-값 쌍 형태로 출력
- UTF-8 키는 문자열, 바이너리 키는 Base64로 인코딩

3. 개별 키 조회

```
// 키 파싱 (Base64 또는 문자열)
var  key []byte
if  err = json.Unmarshal([]byte(fmt.Sprintf("\"%s\"", strKey)), &key);  err != n
key = []byte(strKey)
}


res, err := conn.Runtime(npa.ParaTime).Contracts.InstanceStorage(
ctx,
client.RoundLatest,
contracts.InstanceID(instanceID),
key,
)
```

키 조회 특징:

- Base64 인코딩된 키 지원
- 일반 문자열 키 지원
- CBOR 역직렬화 시도 후 원시 데이터 폴백

WASM 컨트랙트 처리

1. 코드 압축

```
Code: contracts.CompressCode(wasmData)
```

압축 처리:

- 자동 압축을 통한 스토리지 효율성
- 네트워크 전송 최적화
- 투명한 압축/해제 과정

2. 코드 덤프 기능

```
codeStorage, err := conn.Runtime(npa.ParaTime).Contracts.CodeStorage(
ctx,
client.RoundLatest,
contracts.CodeID(codeID),
)
cobra.CheckErr(err)

os.Stdout.Write(codeStorage.Code)
```

코드 덤프:

- 업로드된 WASM 바이너리 추출
- 표준 출력으로 직접 출력
- 코드 검증 및 분석 지원

3. ABI 지원

```
fmt.Printf("ABI: %s (sv: %d)\n", code.ABI, code.ABISubVersion)
```

ABI 관리:

- 현재 ABIOasisV1 지원
- 서브버전을 통한 세밀한 버전 관리
- 호환성 보장

정책 관리 및 업그레이드

1. 정책 표시

```
func formatPolicy(policy *contracts.Policy) string {
switch {
case policy.Nobody != nil:
return "nobody"
case policy.Address != nil:
return fmt.Sprintf("address:%s", policy.Address.String())
case policy.Everyone != nil:
return "everyone"
default:
return "[unknown]"
}
```

```
}
```

정책 표시 기능:

- 사용자 친화적 정책 표시
- 주소 기반 정책의 명확한 표시
- 알 수 없는 정책에 대한 안전한 처리

2. 업그레이드 정책 변경

```
tx := contracts.NewChangeUpgradePolicyTx(nil, &contracts.ChangeUpgradePolicy{
ID: contracts.InstanceID(instanceID),
UpgradesPolicy: *upgradesPolicy,
})
```

업그레이드 정책 변경:

- 런타임 중 정책 변경 가능
- 기존 정책 검증
- 권한 확인 후 변경 적용

3. 주소 해결

```
address, _, err := common.ResolveLocalAccountOrAddress(net, policy)
```

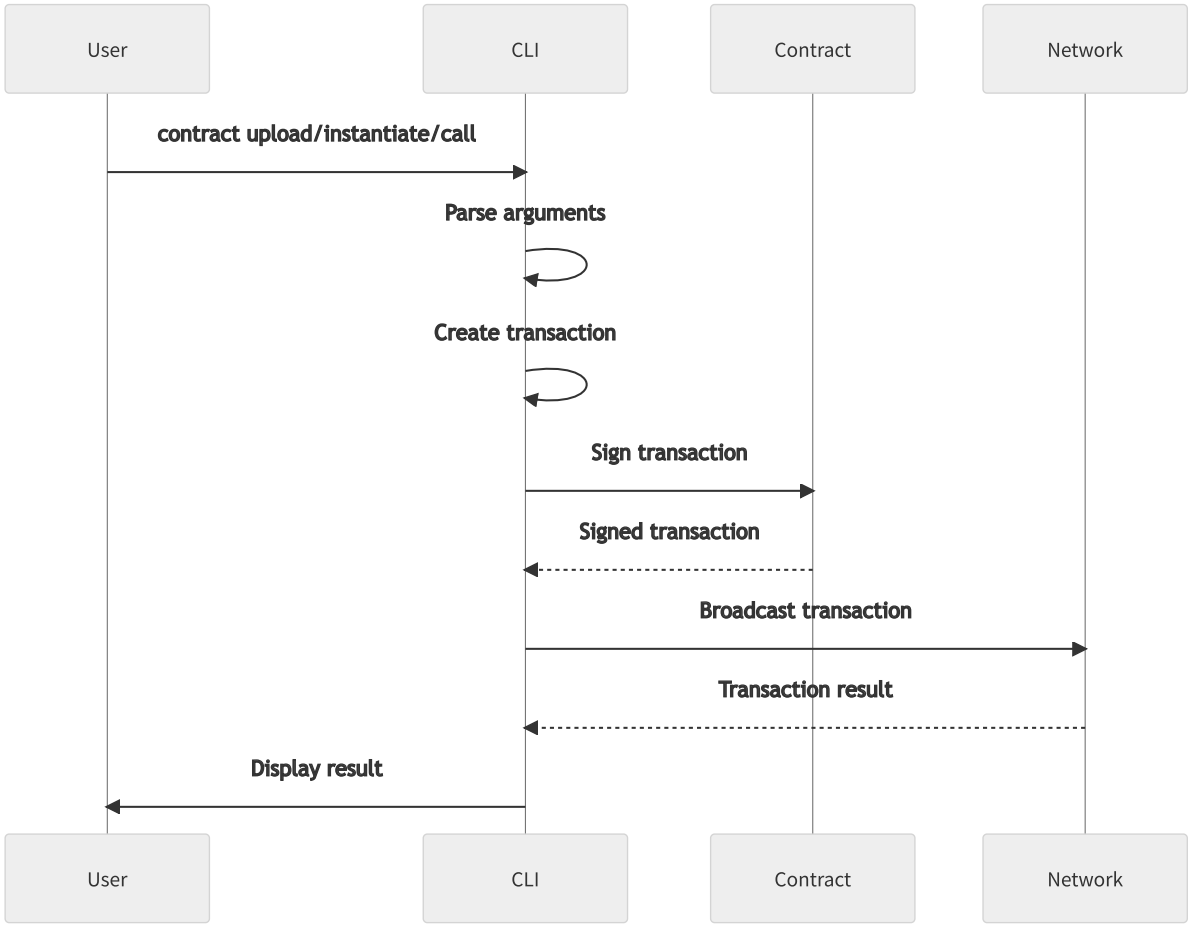
주소 해결 과정:

- 로컬 계정 이름 지원
- 주소록 참조

- 직접 주소 입력 지원

트랜잭션 처리 파이프라인

1. 트랜잭션 생성



2. 가스 추정

```
acc := common.LoadAccount(cfg, npa.AccountName)
```

```
sigTx, meta, err := common.SignParaTimeTransaction(ctx, npa, acc, conn, tx, nil)
```

가스 처리:

- 자동 가스 추정
- 20% 여유분 추가
- 사용자 지정 가스 한도 지원

3. 결과 디코딩

```
decResult, err := callformat.DecodeResult(&rawMeta.Result, meta)
cobra.CheckErr(err)

switch {
case  decResult.IsSuccess():
fmt.Printf("Execution successful.\n")
if  result != nil {
err = cbor.Unmarshal(decResult.Ok, result)
cobra.CheckErr(err)
}
default:
cobra.CheckErr(fmt.Sprintf("Execution failed with error: %s",
PrettyErrorHints(ctx, npa, conn, tx, meta, decResult.Failed)))
}
```

에러 처리 및 디버깅

1. 에러 분류

컨트랙트 관련 에러:

- 코드 업로드 실패
- 인스턴스화 실패
- 호출 실행 실패
- 권한 부족 에러

2. 디버깅 도구

스토리지 검사:

- 컨트랙트 상태 덤프
- 개별 키 조회
- 스토리지 변경 추적

코드 검증:

- WASM 바이너리 덤프
- ABI 호환성 확인
- 코드 해시 검증

3. 사용자 친화적 에러 메시지

```
func PrettyErrorHints(
_ context.Context,
npa *NPASelection,
```

```
_ connection.Connection,
_  interface{},
_  interface{},
failedRes *types.FailedCallResult,
) string {
errMsg := failedRes.Error()
// 컨텍스트별 힌트 추가
return  errMsg
}
```

보안 고려사항

1. 정책 검증

인스턴스화 정책:

- 권한 없는 인스턴스화 방지
- 주소 기반 접근 제어
- 정책 변경 권한 관리

2. 코드 검증

WASM 코드:

- 코드 해시 검증
- ABI 호환성 확인
- 악성 코드 방지

3. 스토리지 보안

접근 제어:

- 공개/기밀 스토리지 구분
- 권한 기반 접근
- 데이터 암호화 지원

성능 최적화

1. 코드 압축

```
Code: contracts.CompressCode(wasmData)
```

최적화 기법:

- 자동 압축을 통한 스토리지 절약
- 네트워크 전송 최적화
- 캐싱 지원

2. 배치 처리

스토리지 덤프:

- 페이지네이션을 통한 메모리 효율성
- 대용량 스토리지 처리
- 점진적 로딩

3. 연결 재사용

네트워크 최적화:

- 연결 풀링
- 요청 배치 처리
- 타임아웃 관리

확장성 설계

1. 새로운 ABI 지원

```
ABI: contracts.ABIOasisV1,
```

확장 방안:

- 새로운 ABI 버전 추가
- 하위 호환성 유지
- 점진적 마이그레이션

2. 추가 스토리지 타입

확장 가능성:

- 새로운 스토리지 종류
- 커스텀 인덱싱
- 고급 쿼리 지원

3. 고급 정책 시스템

정책 확장:

- 다중 서명 정책
- 시간 기반 정책

- 조건부 정책

이 스마트 컨트랙트 시스템은 WebAssembly 기반의 완전한 컨트랙트 생명주기 관리를 제공하며, 보안성, 확장성, 사용자 편의성을 모두 고려한 설계로 구성되어 있습니다.